FAST NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

FAST — NUCES, LAHORE CAMPUS

Department of Cyber Security

CYBER SECURITY (CY2004)

Course Lab Manual

LAB 02

Linux Shell Fundamentals II: I/O Redirection, Users & Processes, Command Chaining, Regular Expressions & Bash Scripting

Prepared by

Teacher Assistant: Muhammad Annus Shahzad

Roll No: 23L-3004

For BS (Cyber Security) students of 3rd Semester

Supervised by

Dr. Arshad Ali, HoS, FSC, NUCES, Lahore

Lab Information Sheet

Field	Details
Course Title	Cyber Security (CY-2004)
Semester	3rd Semester
Lab Number	Lab 02
Lab Title	Linux Shell Fundamentals II — Redirection, Users, Processes, Chaining, Regex & Bash Scripting
Prerequisite	Lab 01 — Deploying a Kali Linux Virtual Machine & Basic Linux Navigation
Estimated Duration	3 Hours (Instructor-Led) + Self-Practice
Environment	Kali Linux VM (VirtualBox / VMware / UTM) deployed in Lab 01
Institute	FAST — National University of Computer and Emerging Sciences, Lahore Campus
Prepared By	Muhammad Annus Shahzad (23L-3004)
Supervised By	Prof. Dr. Arshad Ali

Table of Contents

Lab Information Sheet..... 2
Table of Contents..... 3
1. Introduction.....
5 2. Learning Objectives.....
5 3. Required Resources.....
6 4. Part A — Shell Fundamentals Recap .....
6
4.1 What Is a Shell?..... 6
4.2 Common Shell Types..... 6
4.3 The Shebang Line..... 6
4.4 Login vs. Interactive vs. Non-Interactive Shells..... 7
5. Part B — The Filesystem Hierarchy and Paths..... 7
5.1 Absolute vs. Relative Paths..... 7
5.2 Special Path Symbols 7
5.3 Core Navigation Commands 7
5.4 The Linux Filesystem Hierarchy Standard (FHS) — Key Directories..... 8
6. Part C — Standard Streams and File Redirection.....
8 6.1 Redirection Operators 9
Worked Examples..... 9 6.2
The tee Command..... 9 7.
Part D — Pipelines and Command Chaining 10
7.1 Control (Chaining) Operators..... 10
Worked Examples..... 10 8.
Part E — Users, Groups, and File Permissions 11
8.1 Identifying Users..... 11
8.2 Managing Users and Groups..... 11
8.3 Key System Files 12 8.4
File Permissions..... 12 8.5
Changing Permissions and Ownership ..... 12 9. Part F
— Process Management..... 13 9.1
Viewing Processes..... 13 9.2
Job Control..... 14 9.3
Terminating Processes: Signals..... 14 9.4
The /proc Virtual Filesystem..... 15 10.
Part G — Regular Expressions and Text Processing 15
10.1 Core Regex Metacharacters..... 15
10.2 grep — Searching Text with Patterns 15
10.3 sed — Stream Editor..... 16
10.4 awk — Pattern-Directed Text Processing 16
10.5 Other Essential Text-Processing Utilities..... 16
11. Part H — Digesting Linux Documentation.....
17 11.1 man — Manual Pages..... 17
11.2 Other Documentation Tools 17
12. Part I — Introduction to Bash Scripting.....
18 12.1 Creating and Running Your First Script..... 18
12.2 Variables 18
12.3 Command Substitution and Arithmetic 18
12.4 Positional Parameters (Script Arguments)..... 19

12.5 Conditionals..... 19
12.6 Loops..... 19
12.7 The case Statement 20
12.8 Functions..... 20

12.9 Exit Codes..... 21
12.10 A Complete, Practical Example 21
13. Part J — A Cybersecurity-Relevant Command-Line Toolbox.....
22 13.1 find and xargs.....
22 13.2 Networking Utilities..... 22
13.3 File Integrity and Identification..... 23
13.4 History, Auditing, and Housekeeping 23
14. Part K — Hands-On Lab Exercises .....
23 Exercise 1 — Redirection and Piping .....
23 Exercise 2 — Users, Groups, and Permissions..... 24
Exercise 3 — Process Management..... 24
Exercise 4 — Regex and Text Processing 24
Exercise 5 — Bash Scripting 24
15. Self-Evaluation.....
24 15.1 Self-Check Skills Checklist .....
25 15.2 Conceptual Review Questions .....
25 15.3 Practical Self-Test Tasks..... 25
16. Lab Deliverables and Assessment Rubric .....
26 17. References and Further Reading..... 26

1. Introduction

In Lab 01, you deployed a pre-built Kali Linux virtual machine and took your first steps inside a Linux terminal — logging in, gaining root privileges with sudo, and using keyboard shortcuts such as history recall and tab completion. Every serious cybersecurity task — from penetration testing to log forensics, malware triage, and system hardening — is performed through the command line, because graphical tools are slow, hard to automate, and often unavailable on the servers and appliances that security professionals actually work on.

This lab builds directly on that foundation. You will move from simply *running* commands to truly *understanding* how the shell works underneath: how programs read and write data through file descriptors, how the operating system tracks who is running what, and how you can combine small, single-purpose tools into powerful pipelines. By the end of this lab, you will also be able to write your own Bash scripts — the same technique used to automate reconnaissance, log parsing, and incident-response playbooks in the real world.

Why this matters: Nearly every industry certification relevant to this degree (CompTIA Linux+, CompTIA Security+, OSCP, GIAC GCIH) assumes fluency with exactly the material in this lab. Mastering it now will make every later lab in this course — network scanning, log analysis, exploitation, and forensics — dramatically easier.

2. Learning Objectives

Upon successful completion of this lab, students will be able to:

- Explain the role of the shell as a command interpreter and identify common shell types. • Differentiate between absolute and relative paths and navigate the Linux filesystem hierarchy confidently. • Explain the three standard I/O streams (stdin, stdout, stderr) and redirect or append them to files.
- Combine multiple commands using pipes (|) and control operators (;, &&, ||, &) to build efficient one-line workflows.
- Manage Linux users, groups, and file permissions (symbolic and octal notation) and apply the principle of least privilege.
- Monitor, prioritize, and terminate processes using ps, top, jobs, and the kill family of commands.
- Write and apply regular expressions with grep, sed, and awk to search and transform text — a core skill for log analysis.
- Independently read and interpret manual pages (man) and other built-in Linux documentation. • Write, save, and execute Bash shell scripts containing variables, conditionals, loops, and functions.
- Identify and use command-line utilities that are directly relevant to cybersecurity work (find, xargs, hashing utilities, network utilities, and more).

3. Required Resources

- A host computer with at least 4 GB RAM and 20 GB free disk space.
- Oracle VirtualBox, VMware Workstation/Player, or UTM (Apple Silicon) with the Kali Linux VM created in Lab 01.
- Internet access (for package updates and documentation lookups).
- A working knowledge of the topics covered in Lab 01 (booting the VM, logging in as user kali, and opening a terminal).

Tip: Before starting, take a fresh VirtualBox/VMware snapshot of your Kali VM. Several exercises in this lab (user creation, process termination) modify system state — a snapshot lets you experiment freely and roll back if needed.

4. Part A — Shell Fundamentals Recap

4.1 What Is a Shell?

A **shell** is a command-line interpreter: a program that reads the text you type, parses it, and asks the operating system's kernel to carry it out. It sits between you and the kernel, translating human-readable commands such as `ls -la` into system calls the kernel understands. Every terminal window you open runs an instance of a shell.

4.2 Common Shell Types

Shell	Binary	Notes
Bourne Again Shell	`/bin/bash`	Default on Kali Linux and most Linux distributions; the focus of this lab.
Z Shell	`/bin/zsh`	Default on modern macOS; highly configurable, popular with Kali power users.
Bourne Shell	`/bin/sh`	The original POSIX shell; many system scripts target this for portability.
Dash	`/bin/dash`	A lightweight POSIX-compliant shell used by Debian/Kali for fast boot scripts.
C Shell / TC Shell	`/bin/csh, /bin/tcsh`	C-like syntax; rarely used for scripting today.

To find out which shell you are currently using, run:

```
echo $SHELL
ps -p $$
```

4.3 The Shebang Line

Every executable script should begin with a **shebang** (short for "hash-bang") on its very first line. It tells the kernel which interpreter should run the file:

```
#!/bin/bash
```

Using `#!/usr/bin/env bash` instead is considered more portable, because it locates `bash` using the user's `PATH` rather than assuming a fixed location.

4.4 Login vs. Interactive vs. Non-Interactive Shells

- **Login shell:** Started when you log in (console, SSH). Reads /etc/profile and ~/.bash_profile or ~/.profile. •

Interactive non-login shell: A new terminal window/tab on an already logged-in desktop. Reads ~/.bashrc. •

Non-interactive shell: Used to run a script; reads $BASH_ENV if set, otherwise minimal startup files.

5. Part B — The Filesystem Hierarchy and Paths

Linux organizes everything — files, directories, and even devices — into a single tree rooted at `/`. Understanding how to express the *location* of a file (its **path**) correctly is essential before you can redirect output, write scripts, or reference tools reliably.

5.1 Absolute vs. Relative Paths

Type	Starts With	Example	Meaning
Absolute path	`/`	`/home/kali/Documents/report.tx t`	Full path from the root of the filesystem — always unambiguous.
Relative path	`a name, . or ..`	`Documents/report.txt`	Interpreted relative to the current working directory.

5.2 Special Path Symbols

Symbol	Meaning
`.`	The current directory.
`..`	The parent directory.
~	The current user's home directory (e.g. /home/kali).

`~username`	Another user's home directory.
`-`	The previous working directory (used with cd -).
`/`	The root directory of the whole filesystem.

5.3 Core Navigation Commands

Command	Purpose
`pwd`	Print the full absolute path of the current working directory.
`cd <path>`	Change directory. cd alone (or cd ~) returns to the home directory.
`ls -la`	List all files (including hidden dotfiles) in long format with permissions.
`tree -L 2`	Display a directory as a visual tree, limited to 2 levels deep.
`file <name>`	Identify a file's type by inspecting its contents, not just its extension.
`realpath <name>`	Resolve a relative path or symlink to its full absolute path.
`which / whereis <cmd>`	Locate the executable that will run for a given command name.

5.4 The Linux Filesystem Hierarchy Standard (FHS) — Key Directories

Directory	Contents Relevant to Cybersecurity
`/etc`	System-wide configuration files, including /etc/passwd, /etc/shadow, /etc/sudoers.
`/var/log`	System and application log files — the first place forensic analysts look.
`/home`	Personal directories for each non-root user.
`/root`	The home directory of the root superuser.

Directory	Contents Relevant to Cybersecurity
`/tmp`	World-writable temporary storage; a common malware drop location to monitor.
`/usr/bin, /bin`	Locations of most installed executable programs.
`/proc`	A virtual filesystem exposing live kernel and process information.

`/dev`	Device files representing hardware (disks, terminals, /dev/null, /dev/random).

6. Part C — Standard Streams and File Redirection

Every process in Linux is automatically given three open communication channels, called **standard streams**, identified by small integers known as **file descriptors**:

Stream	File Descriptor	Default Source/Destination
stdin (standard input)	`0`	The keyboard
stdout (standard output)	`1`	The terminal screen
stderr (standard error)	`2`	The terminal screen



Figure 6.1 — The three standard I/O streams that every process inherits.

Separating `stdout` from `stderr` is deliberate: it lets you capture a program's real output while errors still appear on screen (or vice-versa) — critical when scripting security tools such as scanners, where you may want results saved to a file but warnings printed live.

6.1 Redirection Operators

Operator	Effect
`> file`	Redirect stdout to file, overwriting it.
`>> file`	Redirect stdout to file, appending to existing content.
`< file`	Feed file's contents into a command's stdin.
`2> file`	Redirect stderr only to file.
`2>> file`	Append stderr only to file.

`2>&1`	Redirect stderr to wherever stdout currently points (order matters!).
`&> file / >& file`	Redirect both stdout and stderr to file (Bash shorthand).

Operator	Effect
`&>> file`	Append both stdout and stderr to file.
`command << EOF ... EOF`	Here-document: feed multiple lines of literal text as stdin.
`command <<< "text"`	Here-string: feed a single line of text as stdin.

Worked Examples

Save the results of an nmap scan to a file, overwriting any previous scan:

```
nmap -sV 192.168.1.1 > scan_results.txt
```

Append a new log entry to a running audit log rather than erasing it:

```
echo "$(date): audit started" >> audit.log
```

Discard noisy output completely by sending it to `/dev/null`, the Linux "black hole" device:

```
ping -c 4 8.8.8.8 > /dev/null
```

Capture both normal output and errors from a vulnerability scan into one log file:

```
./vuln_scanner.sh target.txt > full_run.log 2>&1
```

Common Pitfall: The order of redirections matters. "command > file 2>&1" sends stderr to the same place stdout was just redirected (file). But "command 2>&1 > file" sends stderr to the terminal, because at the moment `2>&1` executes, stdout is still the screen. Always redirect stdout first, then duplicate stderr onto it.

6.2 The tee Command

Sometimes you want to *both* view output live *and* save it to a file. The `tee` command reads from stdin and writes to stdout as well as one or more files:

```
nmap -sV 192.168.1.1 | tee scan_results.txt
cat access.log | grep "404" | tee -a not_found.log
```

7. Part D — Pipelines and Command Chaining

The Linux philosophy is to build small, single-purpose programs and combine them. The **pipe operator** (`|`) connects

the `stdout` of one command directly to the `stdin` of the next, without ever touching disk. This is one of the most powerful ideas in all of computing, and you will use it constantly for log analysis and reconnaissance.



Figure 7.1 — A four-stage pipeline: each command's output becomes the next command's input.

The pipeline shown above finds and counts repeated failed-login lines in a log file:

```
cat access.log | grep "Failed" | sort | uniq -c
```

7.1 Control (Chaining) Operators

Beyond pipes, Bash provides operators to control whether and how successive commands run:

Operator	Name	Behaviour
`;`	Sequential	Run commands one after another, regardless of success or failure.
`&&`	Logical AND	Run the next command only if the previous one succeeded (exit code 0).
`||`	Logical OR	Run the next command only if the previous one failed (non-zero exit code).
`&`	Background	Run the command asynchronously in the background; the shell prompt returns immediately.
`( cmd1; cmd2 )`	Subshell group	Run commands in a child shell; variable changes do not affect the parent shell.
`{ cmd1; cmd2; }`	Current-shell group	Run commands in the current shell (note the required spaces and trailing ;).

Worked Examples

Update package lists only *if*

```
sudo apt update && sudo apt upgrade -y
```

Try a primary tool, and fall back to an alternative only if it fails:

```
curl -s https://target.local || echo "Target unreachable, trying wget..." && wget
https://target.local
```

Run three independent commands regardless of each other's outcome:

```
whoami; pwd; date
```

Launch a long network scan in the background so you can keep working in the same terminal:


```
nmap -p- 10.10.10.10 -oN full_scan.txt &
```

Security Relevance: Chaining operators appear constantly in real attacker and defender tooling — from one-liner reverse-shell droppers (`cmd1 && cmd2`) to defensive health-check scripts that only proceed when a prior check passes. Recognising these patterns is essential when reading logs or reverse-engineering scripts.

8. Part E — Users, Groups, and File Permissions

Linux is a **multi-user** operating system by design, and its permission model is the single biggest line of defence against unauthorised access. Understanding how users, groups, and permissions interact is fundamental to both attacking misconfigured systems and defending your own.

8.1 Identifying Users

Command	Purpose
`whoami`	Print the current effective username.
`id`	Show the current user's UID, GID, and all group memberships.
`who / w`	List who is currently logged in, and what they are doing.
`last`	Show a history of recent logins, read from /var/log/wtmp.
`su - username`	Switch user, loading that user's full login environment.
`sudo command`	Run a single command with root privileges (if permitted by /etc/sudoers).
`sudo -l`	List the commands the current user is permitted to run with sudo.

8.2 Managing Users and Groups

Command	Purpose
`sudo useradd -m -s /bin/bash alice`	Create user alice with a home directory (-m) and Bash as her shell (- s).
`sudo passwd alice`	Set or change alice's password.
`sudo usermod -aG sudo alice`	Add alice to the sudo group (append, don't overwrite, with -a).
`sudo deluser alice`	Delete the user alice (add --remove-home to also delete her home directory).
`sudo groupadd analysts`	Create a new group named analysts.
`groups alice`	List every group alice belongs to.

8.3 Key System Files

- `/etc/passwd` – One line per user account: username, UID, GID, home directory, and login shell. World readable — often the first file enumerated during a privilege-escalation assessment.
- `/etc/shadow` – Stores hashed passwords and password-ageing policy. Readable only by root — a prime target for password-cracking attacks if leaked.
- `/etc/group` – One line per group, listing its members.
- `/etc/sudoers` – Defines exactly which users/groups may run which commands as root. Always edit with visudo, which syntax-checks your changes before saving.

Inspect these files safely (read-only) with:

```
cat /etc/passwd
sudo cat /etc/shadow
sudo visudo
```

8.4 File Permissions

Run `ls -l` on any file and the first column shows a 10-character permission string, e.g. `-rwxr-xr--`. The first character is the file type (`-` for regular file, `d` for directory, `l` for symbolic link); the remaining nine characters are three permission triads for the **owner**, **group**, and **others**.



Figure 8.1 — Anatomy of a Linux permission string and its octal (numeric) equivalent.

8.5 Changing Permissions and Ownership

Command	Purpose
`chmod 754 file.sh`	Set permissions numerically: owner=rwx(7), group=r-x(5), others=r--(4).
`chmod u+x file.sh`	Symbolically add execute permission for the owner (user).
`chmod go-w file.sh`	Symbolically remove write permission from group and others.
`chmod -R 700 dir/`	Recursively (-R) restrict a directory to the owner only.
`chown alice file.sh`	Change the owner of a file to alice.
`chown alice:analysts file.sh`	Change owner to alice and group to analysts in one step.
`chgrp analysts file.sh`	Change only the group owner of a file.
`umask`	View or set the default permission mask applied to newly created files.

Principle of Least Privilege: Never grant more permission than a task requires — e.g. avoid chmod 777, which grants read, write, and execute to everyone and is a classic misconfiguration exploited during privilege-escalation attacks. Prefer the narrowest chmod value that still lets the job get done.

9. Part F — Process Management

A **process** is a running instance of a program. Every process has a unique Process ID (`PID`), a parent process (`PPID`), an owning user, and a current state. Being able to inspect, prioritise, and terminate processes is essential for both performance troubleshooting and incident response (e.g., killing a malicious or runaway process).

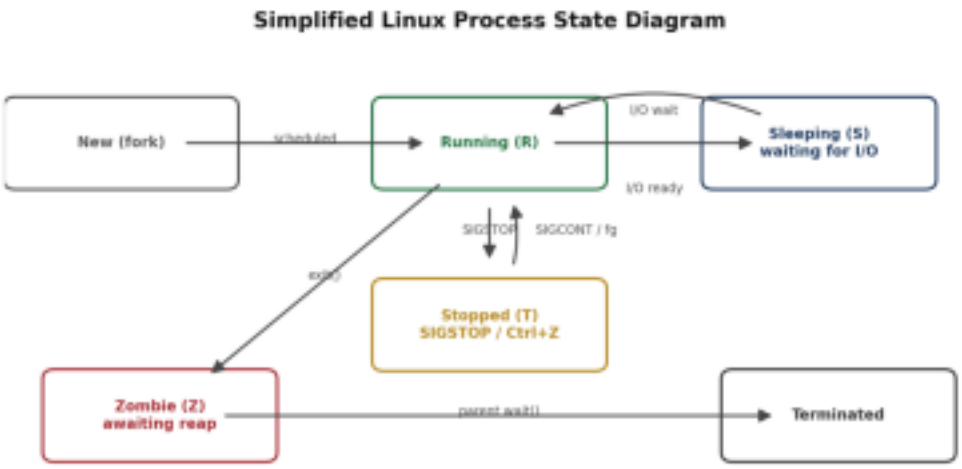


Figure 9.1 — Simplified state transitions a Linux process moves through during its lifetime.

9.1 Viewing Processes

Command	Purpose
`ps`	List processes running in the current terminal session.
`ps aux`	List every process on the system (BSD-style), with user, CPU%, and memory%.
`ps -ef`	List every process (System V style), showing parent PID clearly.
`pstree`	Display processes as a hierarchical parent/child tree.
`top`	Live, auto-refreshing view of processes sorted by resource usage.
`htop`	An enhanced, colourised, interactive alternative to top.
`pgrep -f nmap`	Print the PID(s) of processes whose command line matches nmap.

9.2 Job Control

Command	Purpose
`command &`	Start command in the background.
`jobs`	List background/suspended jobs of the current shell.
`Ctrl+Z`	Suspend (pause) the foreground job, moving it to Stopped state.
`bg %1`	Resume stopped job 1, running it in the background.
`fg %1`	Bring job 1 back to the foreground.
`nohup command &`	Run a command immune to hangup signals, so it survives terminal closure.

`disown`	Detach a background job from the current shell entirely.

9.3 Terminating Processes: Signals

The `kill` family of commands does not simply "kill" a process outright — it *sends a signal* to it, and well-behaved programs choose how to respond. The most important signals are:

Signal	Number	Meaning
`SIGHUP`	1	Hang-up — often used to tell a daemon to reload its configuration.

Signal	Number	Meaning
`SIGINT`	2	Interrupt — what Ctrl+C sends; the process may catch and ignore it.
`SIGKILL`	9	Force-kill immediately; cannot be caught, blocked, or ignored.
`SIGTERM`	15	Polite termination request (the default signal sent by kill); allows cleanup.
`SIGSTOP`	19	Pause the process (like Ctrl+Z); cannot be caught or ignored.
`SIGCONT`	18	Resume a previously stopped process.

Examples:

```
kill 1432 # politely ask PID 1432 (SIGTERM) to stop
kill -9 1432 # force-kill PID 1432 immediately (SIGKILL)
killall firefox # terminate every process named firefox
pkill -f "python3 exploit.py" # kill by matching the command line
```

Best Practice: Always try SIGTERM (the default) before reaching for SIGKILL (-9). SIGTERM gives a process the chance to close files and network connections cleanly; SIGKILL yanks it out instantly and can leave corrupted files or orphaned resources behind.

9.4 The /proc Virtual Filesystem

Linux exposes live kernel and process data as files under `/proc`. Every running process gets its own numbered directory, e.g. `/proc/1432/`, containing files describing its memory maps, open file descriptors, and command line — extremely useful during live forensic investigation.

```
cat /proc/1432/cmdline # the exact command line that launched PID 1432 ls -l
/proc/1432/fd/ # every file descriptor currently open by PID 1432 cat
/proc/cpuinfo # static information about the CPU
cat /proc/meminfo # live memory usage statistics
```

10. Part G — Regular Expressions and Text Processing

Security analysts spend a large share of their time reading text: logs, packet captures exported as text, configuration files, and source code. **Regular expressions** (regex) are a compact language for describing text *patterns* so that tools can search, filter, or rewrite matching text automatically — instead of you scrolling through thousands of lines by eye.

10.1 Core Regex Metacharacters

Symbol	Meaning	Example	Matches
`.`	Any single character	`a.c`	abc, axc, a1c
`*`	Zero or more of the previous item	`ab*c`	ac, abc, abbbc
`+`	One or more (extended regex)	`ab+c`	abc, abbc (not ac)
`?`	Zero or one (extended regex)	`colou?r`	color, colour
`^`	Anchor: start of line	`^Error`	lines beginning with Error
`$`	Anchor: end of line	`failed$`	lines ending with failed
`[]`	Character class (any one of)	`[0-9]`	any single digit

Symbol	Meaning	Example	Matches
`[^]`	Negated character class	`[^0-9]`	any non-digit character
`{n,m}`	Between n and m repetitions	`[0-9]{3}`	exactly 3 digits
`()`	Grouping / capturing	`(ab)+`	ab, abab, abababab
`|`	Alternation (OR)	`cat|dog`	cat or dog
`\d \w \s`	Digit / word char / whitespace (PCRE)	`\d{1,3}`	1 to 3 digits

10.2 grep — Searching Text with Patterns

Option	Effect
`-i`	Case-insensitive search.
`-v`	Invert match — show lines that do NOT match.

`-r / -R`	Recursively search every file inside a directory.
`-n`	Show the line number of every match.
`-c`	Print only a count of matching lines.
`-E`	Use Extended Regular Expressions (enables +, ?, {}, | without escaping).
`-o`	Print only the matched text, not the whole line.
`-l`	Print only the names of files that contain a match.

Practical, security-flavoured examples:

```
grep -i "failed password" /var/log/auth.log
grep -rn "api_key" /var/www/
grep -E "^(1[0-9]{2}|2[0-9]{2})\." access.log # lines starting with a 1xx/2xx status code
grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" access.log # extract every IPv4 address grep -v
"^#" /etc/ssh/sshd_config # show config lines that are not comments
```

10.3 sed — Stream Editor

The `sed` utility applies edits to text as it streams past, most commonly for search-and-replace:

```
sed 's/http/https/' urls.txt # replace first http per line with https sed
's/http/https/g' urls.txt # replace ALL occurrences per line sed -n '10,20p'
access.log # print only lines 10 through 20 sed '/^$/d' notes.txt # delete every blank
line
sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config # edit the file in place
```

10.4 awk — Pattern-Directed Text Processing

Where `grep` finds lines and `sed` edits text, `awk` excels at working with columns of data:

```
awk '{print $1}' access.log # print only the first column (usually the source IP) awk -F:
'{print $1}' /etc/passwd # split on ':' and print the username field awk '$9 == 404 {print $1}'
access.log # print IPs whose 9th field (status code) is 404
```



```
awk '{sum += $1} END {print sum}' bytes.txt # sum a column of numbers
```

10.5 Other Essential Text-Processing Utilities

Command	Purpose
`cut -d: -f1 /etc/passwd`	Extract the 1st field from each line, using : as the delimiter.
`sort`	Sort lines alphabetically; use -n for numeric, -r to reverse.
`uniq -c`	Collapse adjacent duplicate lines, prefixing each with a count (input must be sorted first).

`wc -l`	Count lines (use -w for words, -c for bytes).
`tr 'a-z' 'A-Z'`	Translate/transliterate characters, e.g. convert to uppercase.
`head -n 20 / tail -n 20`	Show the first / last 20 lines of a file.
`tail -f`	Follow a growing file live — essential for watching a log during an attack.
`diff file1 file2`	Show line-by-line differences between two files.

Putting it together: grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" access.log | sort | uniq -c | sort -rn | head -10 extracts every IP address from a web log, then reports the ten most frequent visitors — a one-line reconnaissance query you will reuse throughout this course.

11. Part H — Digesting Linux Documentation

No security professional memorises every flag of every tool. The single most valuable skill in this lab may be learning to read Linux's *built-in* documentation quickly and confidently, rather than depending on the internet.

11.1 man — Manual Pages

`man grep`
`man 5 passwd # section 5 = file formats, disambiguates from section 1 (commands)` Manual pages are grouped into numbered sections:

Section	Covers
1	User commands (e.g. ls, grep, chmod)
2	System calls (kernel-level functions)
5	File formats and configuration file layouts (e.g. passwd)
8	System administration commands (e.g. iptables, useradd)

- NAME — a one-line description of the command.
- SYNOPSIS — the command's exact syntax; [brackets] mean optional, | means choose one.
- DESCRIPTION — a full explanation of behaviour.
- OPTIONS — every flag the command accepts, explained individually.
- EXAMPLES — sample invocations (not always present).

- SEE ALSO — related commands worth exploring.

Inside a man page: press Space/b to page down/up, / to search forward, n to jump to the next match, and q to quit.

11.2 Other Documentation Tools

FAST NUCES Lahore — Cyber Security — Lab 02

Command	Purpose
`command --help`	Print a short usage summary directly from the program itself.
`info command`	Open the (often more detailed) GNU info manual, where available.
`whatis command`	Print the one-line NAME summary from the man page.
`apropos keyword`	Search all man page descriptions for a keyword (equivalent to man -k).
`tldr command`	Community-maintained, example-first "cheat sheet" (install: sudo apt install tldr).

12. Part I — Introduction to Bash Scripting

A **shell script** is simply a text file containing a sequence of shell commands, saved so it can be run repeatedly. Scripting is what turns manual, error-prone, one-off commands into repeatable, shareable automation — the same technique used to build reconnaissance scripts, log-parsers, and automated hardening checklists.

12.1 Creating and Running Your First Script

```
#!/bin/bash
# hello.sh - my first Bash script
echo "Hello, $(whoami)! Today is $(date +%A)."
```

Make it executable and run it:

```
chmod +x hello.sh
./hello.sh
# or, without making it executable:
bash hello.sh
```

12.2 Variables

```
name="Alice" # no spaces around =
echo "Hello, $name" # use $ to read a variable
echo "Hello, ${name}!" # braces avoid ambiguity next to other text
readonly PI=3.14159 # a constant that cannot be reassigned
unset name # delete a variable
```

Quoting rule: Double quotes ("$var") allow variable expansion; single quotes ('$var') treat everything literally. Always double-quote variables that might contain spaces or be empty, to avoid word-splitting bugs.

12.3 Command Substitution and Arithmetic

```
files=$(ls | wc -l) # capture a command's output into a variable echo "There
are $files files here."
total=$((5 + 3)) # arithmetic expansion
let count++ # increment a variable
```

12.4 Positional Parameters (Script Arguments)

Variable	Meaning

Variable	Meaning
`$0`	The script's own name/path.
`$1, $2, ...`	The 1st, 2nd, ... argument passed to the script.
`$#`	The total number of arguments supplied.
`$@`	All arguments, individually quoted — nearly always what you want.
`$?`	The exit status of the last command run (0 = success).
`$$`	The PID of the current shell/script.

```
#!/bin/bash
# usage: ./greet.sh <name> <role>
echo "Script name : $0"
echo "Argument 1 : $1"
echo "Argument 2 : $2"
echo "Total args : $#"
```

12.5 Conditionals

```
#!/bin/bash
read -p "Enter a port number: " port

if [ "$port" -eq 22 ]; then
 echo "SSH port detected."
elif [ "$port" -lt 1024 ]; then
 echo "This is a well-known (privileged) port."
else
 echo "This is a registered/dynamic port."
```

```
fi
```

Bash's `test` (a.k.a. `[`) operators most used in security scripting:

Operator	True when...
`-eq / -ne`	Numbers are equal / not equal
`-lt / -gt / -le / -ge`	Less than / greater than / ≤ / ≥
`== / !=`	Strings are equal / not equal
`-z / -n`	String is empty / non-empty
`-f / -d`	Path exists as a regular file / a directory
`-x / -w / -r`	File is executable / writable / readable by the current user
`&& / ||`	Logical AND / OR between test expressions

12.6 Loops

for loop — iterate over a fixed list, e.g. every IP in a target file:

```
for ip in $(cat targets.txt); do
 echo "Pinging $ip..."
 ping -c 1 "$ip" > /dev/null && echo "$ip is UP" || echo "$ip is DOWN"
```



```
done
```

while loop — repeat as long as a condition holds, e.g. read a file line-by-line:

```
while read -r line; do
 echo "Processing: $line"
done < accounts.txt
```

until loop — repeat until a condition becomes true (opposite of while):

```
count=1
until [ "$count" -gt 5 ]; do
 echo "Attempt number $count"
 count=$((count + 1))
done
```

A C-style `for` loop is also available:

```
for (( i=1; i<=10; i++ )); do
 echo "Scanning host 192.168.1.$i"
done
```

12.7 The case Statement

```
#!/bin/bash
read -p "Choose an action (scan/report/exit): " action
case "$action" in
 scan)
 echo "Starting scan..." ;;
 report)
 echo "Generating report..." ;;
 exit)
 echo "Goodbye."; exit 0 ;;
 *)
 echo "Unknown option: $action" ;;
esac
```

12.8 Functions

```
#!/bin/bash
check_port() {
 local host=$1
 local port=$2
 if timeout 1 bash -c "echo > /dev/tcp/$host/$port" 2>/dev/null; then  echo
"$host:$port is OPEN"
 else
 echo "$host:$port is CLOSED"
 fi
}

check_port "192.168.1.1" 22
```



```
check_port "192.168.1.1" 443
```

Note the `local` keyword — it confines a variable to the function, preventing it from leaking into (or colliding with) the wider script.

12.9 Exit Codes

Every command and script returns a numeric **exit code** to the shell: `0` always means success; any non-zero value (`1`–`255`) signals some kind of failure. Well-written scripts set this deliberately with `exit N`, so that they can be safely chained with `&&`/`||` or checked by other automation.

```
#!/bin/bash
if [ ! -f "$1" ]; then
 echo "Error: file not found." >&2
 exit 1
fi
echo "File found, continuing..."
exit 0
```

12.10 A Complete, Practical Example

The script below combines everything covered in this section: arguments, a function, a loop, a conditional, and a proper exit code — to check whether a list of hosts respond to ping.

```
#!/bin/bash
# host_check.sh - ping every host listed in a file and report status
```

```
# usage: ./host_check.sh hosts.txt

if [ $# -ne 1 ]; then
 echo "Usage: $0 <hostfile>" >&2
 exit 1
fi

hostfile="$1"

if [ ! -f "$hostfile" ]; then
 echo "Error: $hostfile does not exist." >&2
 exit 2
fi

is_alive() {
 ping -c 1 -W 1 "$1" > /dev/null 2>&1
}

up_count=0
down_count=0

while read -r host; do
 [ -z "$host" ] && continue # skip blank lines
 if is_alive "$host"; then
 echo "[UP] $host"
 up_count=$((up_count + 1))
 else
 echo "[DOWN] $host"
 down_count=$((down_count + 1))
```



```
 fi
done < "$hostfile"

echo "-----------------------------" echo
"Summary: $up_count up, $down_count down" exit 0
```


13. Part J — A Cybersecurity-Relevant Command-Line Toolbox

The utilities below build directly on this lab's material and are used constantly throughout the remainder of this course, particularly in reconnaissance, log analysis, and forensics labs.

13.1 find and xargs

Command	Purpose
`find / -perm -4000 2>/dev/null`	Locate every SUID binary on the system — a classic privilege escalation hunting technique.
`find / -mtime -1 2>/dev/null`	Find files modified in the last 24 hours (useful after a suspected compromise).

`find . -name "*.log"`	Recursively find all files ending in .log below the current directory.
`find . -type f -name "*.sh" -exec chmod +x {} \;`	Find every .sh file and make it executable, one at a time.
`find . -name "*.tmp" | xargs rm -f`	Pipe found filenames into xargs to build and run a batch rm command.

`xargs` converts lines of stdin into command-line arguments, letting non-pipe-aware commands like `rm` or `chmod` work efficiently at the end of a pipeline.

13.2 Networking Utilities

Command	Purpose
`ip a`	Show all network interfaces and their assigned IP addresses (modern replacement for ifconfig).
`ss -tulwn`	List listening TCP/UDP ports and the processes bound to them (modern replacement for netstat).
`curl -I https://example.com`	Fetch only the HTTP response headers of a URL.
`wget https://example.com/file`	Download a file directly from the command line.
`dig example.com`	Query DNS records for a domain.
`traceroute example.com`	Show the network hops between you and a destination host.

13.3 File Integrity and Identification

Command	Purpose
`sha256sum file.iso`	Compute a SHA-256 hash — used to verify a file has not been tampered with.
`md5sum file.iso`	Compute an MD5 hash (fast, but cryptographically broken — avoid for security guarantees).

Command	Purpose
`file suspicious.bin`	Identify a file's true type from its contents (magic bytes), regardless of its extension.
`strings suspicious.bin`	Extract human-readable text embedded inside a binary — a first step in basic malware triage.

`stat file.txt`	Show detailed metadata: size, permissions, and access/modify/change timestamps.

13.4 History, Auditing, and Housekeeping

Command	Purpose
`history`	List previously executed commands in this session.
`history | grep ssh`	Search your own command history for a keyword.
`lsattr / chattr +i file`	View / set special filesystem attributes, e.g. making a file immutable so even root cannot modify it without first clearing the flag.
`crontab -l`	List the current user's scheduled (cron) jobs — attackers sometimes hide persistence here.
`df -h / du -sh *`	Show disk space free / used, by filesystem or by item respectively.

Chapter Synthesis: A realistic one-liner combining this entire lab: find /var/log -name "*.log" -mtime -1 | xargs grep -il "failed password" locates every log file modified today, then reports which of those files mention failed logins — using find, xargs, and grep together in a single pipeline.

14. Part K — Hands-On Lab Exercises

Complete the following tasks inside your Kali Linux VM, in order. Take a screenshot of each numbered task's terminal output for your lab submission unless your instructor specifies otherwise.

Exercise 1 — Redirection and Piping

1. Create a directory named lab2_work in your home folder and cd into it.
2. Run ls -la / > root_listing.txt and confirm the file was created.
3. Run ls -la /nonexistent_folder 2> errors.txt and inspect errors.txt.
4. Run a single command that appends both stdout and stderr of ls -la /etc /nofolder to a file named combined.log.
5. Use a pipeline to count how many files exist directly inside /etc (hint: ls | wc -l).

Exercise 2 — Users, Groups, and Permissions

6. Create a new user named secanalyst with a home directory and Bash as the default shell.
7. Set a password for secanalyst using sudo passwd.
8. Create a group named soc_team and add secanalyst to it.
9. Create a file named incident_report.txt, then use chmod to give the owner full access, the group read/write, and others no access at all. Verify with ls -l.

10. Use chown to change the file's group ownership to soc_team.

Exercise 3 — Process Management

11. Launch a long-running background command: ping 8.8.8.8 > ping_test.log &
12. Use jobs and ps aux | grep ping to locate it.
13. Note its PID, then terminate it gracefully with kill <PID>.
14. Start it again, and this time force-kill it using kill -9.
15. Open top (or htop if installed), identify the three processes using the most CPU, and record their names.

Exercise 4 — Regex and Text Processing

16. Download or locate a sample log file (your instructor may provide auth.log or you may use /var/log/auth.log if it exists and is readable).
17. Use grep -c to count how many lines contain the word "session".
18. Use grep -oE to extract every IPv4 address that appears in the file.
19. Pipe that output through sort | uniq -c | sort -rn to rank IP addresses by

frequency. 20. Use awk to print just the first column (timestamp/date) of every line.

Exercise 5 — Bash Scripting

21. Create a script named port_report.sh that accepts a hostname and a list of ports as arguments.
22. Inside the script, write a function is_open() that uses /dev/tcp to test whether a given port is reachable (see section 12.8 for the technique).
23. Loop over every port argument (hint: use shift or iterate over $@ after the first argument) and call is_open for each.
24. Print a clean summary at the end showing how many ports were open vs. closed, and exit with code 0 on success.
25. Test your script against localhost and at least three common ports (22, 80, 443).

15. Self-Evaluation

Use the checklist and questions below to honestly assess your own understanding *before* submitting this lab. If you cannot confidently answer a question, revisit the corresponding section rather than guessing.

15.1 Self-Check Skills Checklist

☐ I can explain the difference between stdout and stderr, and why keeping them separate is useful. ☐ I can redirect output to a new file, append to an existing file, and merge stderr into stdout correctly. ☐ I can explain the difference between a pipe (|) and the && / || / ; control operators. ☐ I can read a permission string like -rwxr-xr-- and convert it to and from octal notation (e.g. 754). ☐ I can create a user, create a group, and assign a user to a group.

☐ I can find a running process by name and terminate it using an appropriate signal.

☐ I can write a grep command using at least three different regex metacharacters.

☐ I can explain what sed and awk are each best suited for, and how they differ from grep. ☐ I can look up an unfamiliar command's usage using man or --help without searching the internet.

☐ I can write a Bash script containing a variable, a conditional, a loop, and a function, and can explain what $1, $@, $#, and $? each mean.

15.2 Conceptual Review Questions

26. Why does Linux separate stdout and stderr into two different streams instead of merging them by default?
27. What is the practical difference between command > file 2>&1 and command 2>&1 > file? Why does the order matter?
28. Explain, in your own words, the security risk of a file with permission mode 777 versus 750. 29. Why is /etc/shadow readable only by root, while /etc/passwd is readable by everyone?
30. What is the difference between SIGTERM and SIGKILL, and when would a system administrator prefer one over the other?
31. Give an example of a real cybersecurity task where a regular expression would save significant time compared to reading a file manually.
32. Why should the local keyword be used for variables declared inside a Bash function?
33. Explain what a non-zero exit code communicates to the rest of a script, and how && and || make use of it.

15.3 Practical Self-Test Tasks

Complete these independently, without referring back to the lab content, to gauge true retention: 34. Write a single command that lists every file in /var/log larger than 1 MB, sorted from largest to smallest.

35. Write a command pipeline that prints only the unique usernames listed in /etc/passwd, one per line, sorted alphabetically.
36. Write a 5-line Bash script that asks the user for a filename and prints "File exists" or "File not found" accordingly.
37. Without looking it up, write the chmod octal value that grants the owner full access, the group read-only access, and others no access at all.
38. Write a kill command that sends SIGKILL to every process whose name matches "stress-test".

Self-Grading Guide: Score yourself out of 10 on the checklist above. 9–10: excellent, proceed to Lab 03. 6–8: solid, but re-read the sections tied to any unchecked boxes. Below 6: repeat the hands-on exercises in Part K before moving on — the remainder of this course assumes fluency with this material.

16. Lab Deliverables and Assessment Rubric

Submit a single PDF containing:

- Terminal screenshots for every task in Exercises 1 through 5 (Section 14).

- The complete source code of port_report.sh from Exercise 5, pasted as text (not just a screenshot).
- Written answers to all eight Conceptual Review Questions (Section 15.2).
- Your completed self-check score from Section 15.1.

Component	Weight	Criteria
Exercise 1 — Redirection & Piping	15%	All redirection/piping tasks completed with correct, verifiable output.

Component	Weight	Criteria
Exercise 2 — Users & Permissions	15%	User/group created correctly; permissions match the specification exactly.
Exercise 3 — Process Management	15%	Correct use of background jobs, job control, and appropriate signals.
Exercise 4 — Regex & Text Processing	20%	Regex patterns are correct and outputs are accurately interpreted.
Exercise 5 — Bash Script	25%	Script runs without errors, uses functions/loops/arguments correctly, and produces an accurate summary.
Conceptual & Self-Evaluation	10%	Answers demonstrate genuine understanding, not memorised definitions.

17. References and Further Reading

- GNU Bash Reference Manual — https://www.gnu.org/software/bash/manual/bash.html
- The Linux Documentation Project — https://tldp.org/
- GNU Core Utilities Manual — https://www.gnu.org/software/coreutils/manual/ • grep, sed, and awk manual pages (man grep, man sed, man awk), accessed locally on Kali Linux. • Filesystem Hierarchy Standard (FHS) 3.0 — https://refspecs.linuxfoundation.org/fhs.shtml • Kali Linux Documentation — https://www.kali.org/docs/
- OverTheWire "Bandit" Wargame (recommended self-practice for shell fluency) — https://overthewire.org/wargames/bandit/